

APPLIED AGENTIC AI FOUNDATION PROGRAMME

Beginner Track • Windows • Terminal-first

MONTH 1 — LAB GUIDE

Build a Rule-Based Chatbot

From an empty computer to a talking chatbot — one very small step at a time.

No prior experience needed. Every task is a single sitting and ends with something that runs.

What you will build this month

- A chatbot you can talk to, that answers using rules *you* write.
- It greets you, understands keywords, answers many topics, and has a personality.
- It starts in the black terminal window, then gets a simple visual window at the end.
- You finish with real Python skills and your very first shipped project on GitHub.

How to use this guide

- Work top to bottom. Do the tasks in order — each one builds on the last.
- Whenever you see a dark box, **type that code exactly** (spelling, colons, and indentation all matter).
- After running, compare your result with the green “*What you should see*” box.
- Don't rush. If something breaks, that's normal — Day 6 is all about fixing errors.

Month 1 at a glance

This month is split into four weeks. Each week has a clear goal, and each day is broken into very small tasks. By the last day you will have a finished chatbot you can show people.

Week	Focus	What you ship
Week 1	Learn Python & get your first bot talking	A bot that chats in the terminal (v0.1)
Week 2	Make the bot smarter (keywords, topics, functions)	A capable keyword bot with personality (v0.2)
Week 3	Memory, personality & saving to GitHub	Your project live on GitHub (v0.3)
Week 4	Give your bot a window & ship	A finished chatbot app with a window (v1.0)

Golden rule: if a step doesn't work, don't panic and don't skip it. Re-read the step slowly, check every character against the code box, and look at Day 6 (reading errors). Ninety percent of beginner problems are a small typo.

Week 1 — Learn Python & get your first bot talking

Goal of the week: install your tools, learn the handful of Python basics you need, and finish with a tiny bot that says hello and replies to you.

Day 1 — Set up your computer

Today you install the two tools every task uses: **Python** (the language) and **VS Code** (where you write code). Nothing to code yet — just setup. Take it slow.

Task 1.1 — Install Python

1. Open your web browser and go to <https://www.python.org/downloads/>.
2. Click the big yellow button that says “**Download Python 3.x.x**”. Save the file.
3. Open the downloaded file (it's in your Downloads folder, at the bottom of the browser).
4. VERY IMPORTANT: at the bottom of the install window, tick the box “**Add python.exe to PATH**” before clicking Install.
5. Click “**Install Now**” and wait for it to finish. Then click Close.

⚠ **If you forget the “Add to PATH” tick**, Python won't work in the terminal later. If unsure, uninstall and reinstall with the box ticked.

Task 1.2 — Check Python installed correctly

6. Press the **Windows key**, type `cmd`, and press Enter. A black window (the “terminal” or “Command Prompt”) opens.
7. Type this exactly and press Enter:

```
python --version
```

✓ **What you should see**

```
Python 3.12.1
```

```
(your numbers may differ slightly – that's fine)
```

8. If you instead see an error, close the terminal, restart your computer, and try again. Still failing? Reinstall Python with the PATH box ticked (Task 1.1, step 4).

Task 1.3 — Install VS Code (your code editor)

9. Go to <https://code.visualstudio.com/> and click the blue Download for Windows button.
10. Open the downloaded file. Accept the agreement and keep clicking Next.
11. On the “Select Additional Tasks” screen, tick “**Add to PATH**” and “**Open with Code**” options, then Install.
12. Launch VS Code when it finishes.

Task 1.4 — Add the Python extension to VS Code

13. In VS Code, click the **Extensions** icon on the left bar (it looks like four squares).
14. In the search box type **Python**.
15. Click the one made by **Microsoft** and press **Install**.

🚩 **Checkpoint:** Python runs in the terminal, and VS Code is installed with the Python extension. You're ready

to write code tomorrow.

Day 2 — Your first Python program

Today you create a project folder, write your first line of code, and run it. This is the loop you'll repeat every single day: write, save, run.

Task 2.1 — Make your project folder

16. On your Desktop, right-click → **New** → **Folder**. Name it **chatbot**.
17. Open VS Code. Choose **File** → **Open Folder** and select your **chatbot** folder.
18. If VS Code asks “Do you trust the authors?”, click **Yes, I trust**.

Task 2.2 — Create your first file

19. In VS Code's left panel, hover over the **chatbot** name and click the **New File** icon (a page with a +).
20. Name the file **hello.py** and press Enter. The **.py** tells the computer it's Python.

Task 2.3 — Write and run “Hello”

21. Click into the empty file on the right and type exactly:

```
print("Hello! I am your first program.")
```

22. Save with **Ctrl + S**.
23. Run it: click the **Run** triangle at the top-right of VS Code. A terminal opens at the bottom.

✓ What you should see

Hello! I am your first program.

What is print()? It shows text on the screen. The words inside the quotes are what appears. Change the words, save, and run again to see it update.

Task 2.4 — Play and break things (on purpose)

24. Change the message to your own sentence. Save, run, confirm it changed.
25. Now remove one quote mark on purpose and run. See the red error? That's called a *SyntaxError*. Put the quote back and it works again. Errors are normal and fixable.

 **Checkpoint:** You can create a file, write code, save, and run it — and you've seen your first error and fixed it.

Day 3 — Variables and getting input

A chatbot needs to remember things (like your name) and listen to what you type. Today: variables (boxes that store values) and input (asking the user a question).

Task 3.1 — Store a value in a variable

26. Create a new file **day3.py**. Type:

```
name = "Alex"
print("Hello " + name)
```

27. Save and run.

✓ What you should see

Hello Alex

Here `name` is a variable — a labelled box holding the text `"Alex"`. The `+` joins two pieces of text together.

Task 3.2 — Ask the user a question

28. Replace everything in the file with:

```
name = input("What is your name? ")
print("Nice to meet you, " + name)
```

29. Save and run. The terminal now waits for you — click into it, type your name, press Enter.

✓ What you should see

What is your name? Sam
Nice to meet you, Sam

`input()` shows your question and waits for the user to type. Whatever they type is stored so you can use it. This is exactly how your chatbot will listen.

Task 3.3 — Combine them into a mini greeter

30. Make the program ask the user's name AND their favourite colour, then print a friendly sentence using both.
31. Hint — you'll need two `input()` lines and one `print()` that joins the pieces with `+`.

✓ What you should see

What is your name? Sam
What is your favourite colour? blue
Hi Sam, blue is a lovely colour!

🚩 **Checkpoint:** You can store values in variables and ask the user for input — the two things a chatbot does constantly.

Day 4 — Making decisions with if / else

A rule-based chatbot is really just a big set of “if the user says X, reply Y” rules. Today you learn the `if / else` statement that powers every one of those rules.

Task 4.1 — Your first if statement

32. New file `day4.py`. Type exactly (the spaces before `print` are important):

```
mood = input("How are you? ")
if mood == "good":
    print("Great to hear!")
else:
    print("I hope your day gets better.")
```

33. Save and run. Type `good` → first reply. Run again, type anything else → second reply.

✓ What you should see

How are you? good

Great to hear!

Three things to notice:

- `==` (two equals) means “is equal to”. One `=` is only for storing in a variable.
- The line ends with a colon `:`
- The reply lines are **indented** (pushed right with 4 spaces). Python uses this spacing to know what belongs to the `if`.

Task 4.2 — Handle more than two cases with `elif`

34. Replace the file with:

```
mood = input("How are you? ")
if mood == "good":
    print("Great to hear!")
elif mood == "sad":
    print("Sorry to hear that. I am here to chat.")
elif mood == "tired":
    print("Get some rest soon!")
else:
    print("Thanks for sharing.")
```

35. Save and run several times, typing good, sad, tired, and something random each time.

`elif` means “else if” — it lets you check as many cases as you like before the final `else` catch-all.

 **Checkpoint:** You can branch your program with `if` / `elif` / `else`. This is the core engine of your chatbot.

Day 5 — Loops: keep the conversation going

Right now your program asks one question and stops. A real chat keeps going until you say bye. Today's tool: the `while` loop, which repeats until you tell it to stop.

Task 5.1 — A loop that repeats

36. New file `day5.py`:

```
while True:
    message = input("You: ")
    print("Bot: I heard you say " + message)
```

37. Save and run. Type anything — it echoes back, then asks again, forever.

✓ **What you should see**

```
You: hello
Bot: I heard you say hello
You: how are you
Bot: I heard you say how are you
You:
```

while True: means “repeat the indented lines again and again.” Great for a chat — but right now it never stops. Next task fixes that.

How to stop a runaway program: click the terminal and press **Ctrl + C**. This force-stops it any time.

Task 5.2 — Let the user quit with “bye”

38. Replace with:

```
while True:
    message = input("You: ")
    if message == "bye":
        print("Bot: Goodbye!")
        break
    print("Bot: I heard you say " + message)
```

39. Save and run. Chat a bit, then type bye — it says goodbye and stops properly.

✓ What you should see

```
You: hi
Bot: I heard you say hi
You: bye
Bot: Goodbye!
```

break immediately exits the loop. Notice “bye” is indented further because it lives inside the **if**, which lives inside the **while**.

 **Checkpoint:** Your program now holds a real back-and-forth conversation and can end cleanly. This is the skeleton of your chatbot.

Day 6 — Reading errors without fear

Everyone hits errors — beginners and experts alike. The skill isn't avoiding them, it's reading them calmly. Today you meet the common ones on purpose so they never scare you.

Task 6.1 — The three errors you'll see most

Create **day6.py** and try each broken snippet below, one at a time. Read the error, then apply the fix.

① **SyntaxError** (a typo in the structure)

```
print("hello)    # missing closing quote
```

Fix: add the missing **"** so it reads **print("hello")**.

② **IndentationError** (wrong spacing)

```
if 1 == 1:
print("yes")    # this line needs 4 spaces in front
```

Fix: put 4 spaces before **print** so Python knows it belongs to the **if**.

③ **NameError** (a misspelled name)

```
message = "hi"
print(msg)    # "message" is misspelled
```

Fix: spell **message** the same way both times.

How to read any error:

- Look at the **last line** — it names the error type.
- Look at the **line number** it mentions — the problem is there or just above.
- Fix one thing, save, run again. Repeat. One error at a time.

🚩 **Checkpoint:** Errors no longer scare you. You can identify the three common types and fix each one.

Day 7 — Ship v0.1: your first talking bot

Time to combine everything from Week 1 into your first real chatbot and save it as version 0.1. It won't be clever yet — that's Week 2 — but it will genuinely chat.

Task 7.1 — Build chatbot.py

40. Create a new file called **chatbot.py** (this becomes your main file for the whole month).
41. Type it out fully — don't copy-paste. Typing it cements the learning:

```
print("Hi! I am ChatBot. What is your name?")
name = input("You: ")
print("Nice to meet you, " + name + "! Type bye to leave.")

while True:
    message = input(name + ": ")
    if message == "bye":
        print("ChatBot: Goodbye, " + name + "!")
        break
    elif message == "hello":
        print("ChatBot: Hello there!")
    elif message == "how are you":
        print("ChatBot: I am just code, but I feel great!")
    else:
        print("ChatBot: I am not sure how to answer that yet.")
```

42. Save and run. Have a full conversation: greet it, ask how it is, then say bye.

✓ What you should see

```
Hi! I am ChatBot. What is your name?
You: Sam
Nice to meet you, Sam! Type bye to leave.
Sam: hello
ChatBot: Hello there!
Sam: bye
ChatBot: Goodbye, Sam!
```

🚩 **Checkpoint:** 🎉 You built a working chatbot from scratch. Week 1 done. Next week you make it much smarter.

Week 2 — Make the bot smarter

Goal of the week: your bot currently only matches exact phrases. This week it will understand keywords anywhere in a sentence, know many topics, and use tidy functions.

Day 8 — Understand keywords, not exact matches

Right now, “hello” works but “oh hello there” doesn't. The word `in` fixes that — it checks whether a word appears *anywhere* in the message.

Task 8.1 — The 'in' keyword

43. In a scratch file, test this idea:

```
message = "oh hello there friend"
if "hello" in message:
    print("Found a greeting!")
```

44. Run it. Even though the message is long, it detects the word hello inside it.

✓ **What you should see**

Found a greeting!

Task 8.2 — Upgrade chatbot.py to use 'in'

45. In `chatbot.py`, change your `==` checks to `in` checks. For example:

```
elif "how are you" in message:
    print("ChatBot: I feel great!")
```

46. Save and run. Now try messy sentences like “hey how are you doing today”. It still understands.

🚩 **Checkpoint:** Your bot understands keywords inside natural sentences, not just exact phrases.

Day 9 — Ignore capital letters

“Hello”, “hello”, and “HELLO” are different to a computer. Today you make your bot treat them all the same using `.lower()`.

Task 9.1 — Lowercasing the message

47. Just after you read the message, add a line to lowercase it:

```
message = input(name + ": ")
message = message.lower()
```

48. Save and run. Now type “HELLO” and “Hello” — both are understood.

✓ **What you should see**

Sam: HELLO

ChatBot: Hello there!

`.lower()` makes every letter small. Doing this once at the top means all your keyword checks can be written in lowercase and just work.

🚩 **Checkpoint:** Capital letters no longer confuse your bot.

Day 10 — Add lots of topics

Today is pure fun: teach your bot to talk about many things. Each topic is just another `elif` with a keyword.

Task 10.1 — Add at least 8 topics

49. Add `elif` blocks for keywords like: `name`, `time`, `weather`, `joke`, `food`, `help`, `bye`. Example:

```
elif "joke" in message:
    print("ChatBot: Why did the programmer quit? They lost their domain!")
elif "help" in message:
    print("ChatBot: You can ask me about the time, jokes, or food.")
```

50. Save and run. Try each topic. Add a few of your own that reflect your personality.

 **Checkpoint:** Your bot can hold a varied conversation across many topics.

Day 11 — Tidy up with functions

Your file is getting long. A function is a reusable named block of code. Today you move the reply logic into one tidy function so the code is clean and easy to grow.

Task 11.1 — Create a `get_reply` function

51. Near the top of `chatbot.py`, define a function that takes a message and returns a reply:

```
def get_reply(message):
    if "hello" in message:
        return "Hello there!"
    elif "joke" in message:
        return "Why did the programmer quit? They lost their domain!"
    else:
        return "I am not sure how to answer that yet."
```

Task 11.2 — Use the function in your loop

52. In the loop, replace the big `if/elif` block with two lines:

```
reply = get_reply(message)
print("ChatBot: " + reply)
```

53. Save and run. It behaves the same — but your code is now far cleaner.

Why functions? They keep code organised and reusable. When you add the visual window in Week 4, you'll reuse this exact `get_reply` function without changing it.

 **Checkpoint:** Your reply logic lives in one clean, reusable function.

Days 12-14 — Level up & polish

- Day 12: Add a `random` reply feature so the bot varies its answers (introduce `import random` and `random.choice([...])`).
- Day 13: Add 5 more topics and a fallback that suggests what the user can ask when it doesn't understand.
- Day 14: Ship v0.2 — save your progress, write down what it can do, and have a friend test it.

 **Checkpoint:** End of Week 2: a genuinely capable keyword chatbot with personality, built from clean functions.

Week 3 — Memory, personality & saving to GitHub

Goal of the week: give your bot a memory within the chat, a stronger personality, and put your project online with Git and GitHub so it's safe and shareable.

Day 15 — Give the bot a short memory

- Store the user's name and refer back to it. Keep a running list of what they've said using a `list` and `.append()`.
- Task: when the user types “what did I say?”, the bot repeats their previous messages back.

Day 16 — Counters and small talk

- Add a counter that tracks how many messages have been sent (a number variable you add `+ 1` to each loop).
- Task: after 5 messages the bot says something like “We've chatted a lot today!”.

Day 17 — Strengthen the personality

- Give your bot a consistent name, tone, and catchphrase. Rewrite several replies in its voice.

Day 18 — Install Git

54. Download Git from <https://git-scm.com/download/win>. Run the installer and accept all the default options (just keep clicking Next).
55. Open a fresh terminal and check it works:

```
git --version
```

✓ What you should see

```
git version 2.4x.x
```

Day 19 — Put your project under Git

56. In VS Code's terminal (make sure it shows your `chatbot` folder), type these one at a time:

```
git init
git add .
git commit -m "My first chatbot"
```

57. Each command runs once. The last one saves a snapshot of your work called a commit.

First time using Git? It may ask you to set your name and email. Just follow the two `git config` commands it prints, then re-run the commit.

Days 20-21 — Push to GitHub & ship v0.3

58. Create a free account at <https://github.com>.
59. Click **New repository**, name it `chatbot`, and click Create. Do NOT tick any add-file options.
60. GitHub then shows a list of commands under “...or push an existing repository”. Copy those lines into your terminal one at a time.
61. Refresh the GitHub page — your code is now online for the world to see.



Checkpoint: Your chatbot is saved, version-controlled, and live on GitHub. That's a real portfolio piece.

Week 4 — Give your bot a window & ship v1.0

Goal of the week: move your bot from the black terminal into a simple graphical window with a text box and a Send button, using Python's built-in `tkinter` toolkit. Then polish and ship the finished version 1.0.

Day 22 — Your first window

62. Create a new file `chatbot_window.py` and type:

```
import tkinter as tk

window = tk.Tk()
window.title("My ChatBot")
label = tk.Label(window, text="Hello! I am your ChatBot.")
label.pack(padx=20, pady=20)
window.mainloop()
```

63. Save and run. A small window appears with your greeting. Close it to stop the program.

✓ What you should see

A window titled "My ChatBot" opens showing: Hello! I am your ChatBot.

`tkinter` comes free with Python — nothing to install. `.pack()` places things in the window, and `mainloop()` keeps the window open.

Day 23 — Add a text box and a Send button

- Add a `tk.Entry` (a box the user types into) and a `tk.Button` labelled Send.
- Connect the button to a function that reads the box and shows a reply — reusing your `get_reply` function from Week 2.

Day 24 — Show the conversation

- Add a `tk.Text` area that displays the whole conversation, adding each new line as you chat.

Day 25 — Make it pretty

- Add colours, a nicer font, and a sensible window size. Make it feel like a real little app.

Days 26-30 — Final polish & ship v1.0

- Day 26: Test everything and fix any bugs. Try to break it, then fix what breaks.
- Day 27: Write a `README.md` explaining what your bot does and how to run it.
- Day 28: Record a short screen video demo of your bot chatting.
- Day 29: Commit and push the final version to GitHub (`git add .` → `git commit -m "v1.0"` → `git push`).
- Day 30: Show someone your finished chatbot. Celebrate — you built and shipped real software.

Month 1 complete

You started with an empty computer and finished with a chatbot that has a window, a personality,

memory, and a home on GitHub. More importantly, you learned to write, run, debug, and ship real Python. Month 2 builds directly on this — see you there.